TUTORIAL | TUTORIAL

Welcome & Setup

1

MLSys·im: First-Principles ML Systems Modeling

A Hands-On Tutorial

Vijay Janapa Reddi

Tutorial

Harvard University

The $200 Million Question

Meta spent $200M training Llama-3-405B.

Before a single GPU was purchased:

- How would you know **16,384 H100s** was the right fleet?
- How would you know **405B parameters** was the right model size?
- How would you know it would take **54 days**, not 540?

We will answer all three questions today — on your laptop, in under a second, with no GPU.

Answer in 0.1 Seconds

```python
import mlsysim

profile = mlsysim.Engine.solve(
    mlsysim.Models.Language.Llama3_8B,
    mlsysim.Hardware.Cloud.H100,
    batch_size=1,
)
print(f"Bottleneck: {profile.bottleneck}")   # Memory
print(f"MFU: {profile.mfu:.3f}")             # 0.003
```

That took 0.1 seconds. On a laptop. No GPU.

Now imagine doing this for every hardware option, every model size, every parallelism strategy, every region. **That is mlsysim.**

What You Will Learn Today

By the end of this tutorial you will be able to:

1. **Identify** which physical constraint is the binding bottleneck for any ML workload on any hardware.
2. **Decompose** training and inference time using the Iron Law.
3. **Compare** hardware configurations quantitatively with `mlsysim`.
4. **Reason** about the compute–memory–communication tradeoff space.
5. **Estimate** TCO and carbon footprint for a real deployment.

All you need is a laptop and `pip install mlsysim`

Setup: Install & Verify

Open a terminal and run:

```
pip install mlsysim
python3 -c "import mlsysim; print(mlsysim.__version__)"
# Expected output: 0.1.0
```

Then run the hello-world sanity check:

```
import mlsysim
model = mlsysim.Models.Language.Llama3_8B
hw    = mlsysim.Hardware.Cloud.H100
prof  = mlsysim.Engine.solve(model, hw, batch_size=1)
print(prof.bottleneck)    # -> "Memory"
```

The 22 Physical Walls of ML Systems

Domain 1: Node

1. Compute Wall
2. Memory Wall
3. Software Wall (MFU)
4. Serving Wall
5. Batching Wall (KV cache)
6. Streaming Wall
7. Tail Latency Wall

Domain 2: Data

Domain 3: Algorithm

11. Complexity Wall (Chinchilla)
12. Reasoning Wall
13. Fidelity Wall (Compression)

Domain 4: Fleet

14. Communication Wall
15. Fragility Wall
16. Multi-Tenant Wall

The Iron Law of ML Systems

$$T = \frac{\text{FLOPs}}{N \times \text{Peak} \times \text{MFU} \times \eta_{\text{scaling}} \times \text{Goodput}}$$

Every wall maps to one of these five denominator terms.
This single equation is our compass for the entire day.

TUTORIAL | TUTORIAL

Related Work

2

The Landscape: Existing ML Performance Tools

Tool	Type	Speed	Scope	HW Coverage	Access
ASTRA-sim	Discrete-event	Hours	Network	NVIDIA	Open
Calculon	Analytical	Seconds	Training	NVIDIA	Open
DeepSpeed Profiler	Runtime	Real-time	Single-node	NVIDIA	Requires GPU
LLMPerf / vLLM	Empirical	Minutes	Serving	NVIDIA, AMD	Requires GPU
Roofline Toolkit	Empirical	Minutes	Compute/memory	Intel, NVIDIA	Requires HW
Internal (Google, Meta)	Analytical	Seconds	Full-stack	Custom	Proprietary
mlsysim	**Analytical**	**Sub-second**	**22 walls**	**5 vendors**	**Open, no GPU**

Key observations: (1) Open tools cover 3–4 constraints each. (2) Most are NVIDIA-only. mlsysim covers **NVIDIA, AMD, Intel, Google, Cerebras,** and custom silicon—full stack from compute to carbon.

What Makes mlsysim Different?

1. **Breadth: 22 walls in one pipeline**
   Others cover 3–4 constraints.
   mlsysim composes all 22 into a single `Pipeline.solve()` call.

2. **Unit safety: Pint dimensional analysis**
   Every quantity carries its unit.
   `GB + TFLOPS` ⇒ DimensionalityError.
   Catches errors spreadsheets never will.

3. **Traceability: Every constant is cited**
   `TraceableConstant` records source, date, and DOI for every hardware spec.

The Trifecta

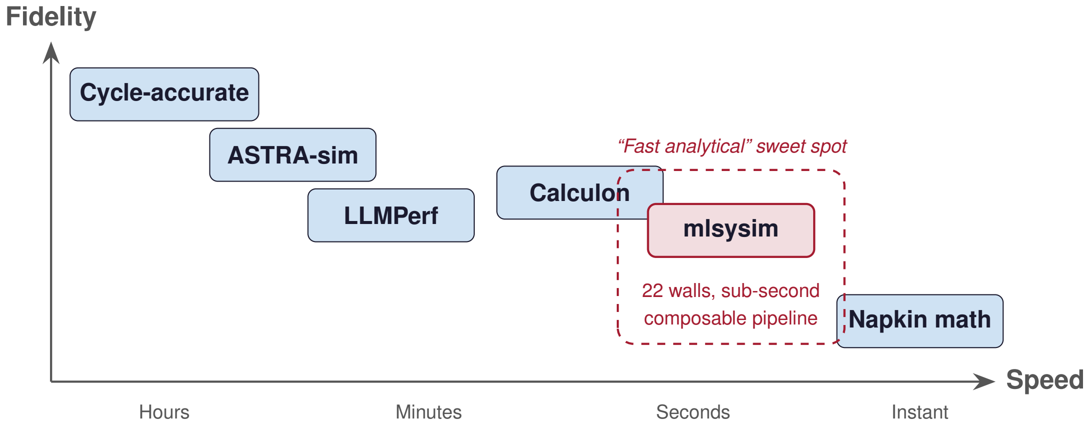
Feature	Others
22 walls	3–4
Unit safety	None
Traceability	Rare

One-Liner

```
pip install mlsysim
```

No GPU. No cluster.
Just Python 3.10+.

The Fidelity–Speed Spectrum



mlsysim trades cycle-accuracy for speed and breadth.

What mlsysim Does NOT Do

Not a replacement for:

- **Benchmarking** — real measurements on real hardware
- **Cycle-accurate simulation** — microarchitectural detail
- **Production deployment** — no orchestration, no serving
- **Framework profiling** — no kernel-level tracing

What it IS:

- A **reasoning framework**
- Answers “which constraint binds?”
- Answers “is this hardware sufficient?”
- Answers “what should I try first?”
- Runs in <1 ms per evaluation
- No GPU required

Prediction accuracy:

Within $2\text{–}5\times$ of measured performance.

“All models are wrong, but some are useful.”

How to Think About Accuracy: The CPI Analogy

Patterson's Insight (Computer Architecture)

Hennessy & Patterson did not predict CPI from first principles.

They **measured** CPI empirically, then used it to **reason** about architectural tradeoffs. The value was in the **framework**, not the exact number.

Our Approach (ML Systems)

We do not predict η (efficiency) from first principles.

We **measure** η empirically (MFU from published papers), then use it to **reason** about system tradeoffs across 22 walls.

	Computer Architecture	ML Systems
Empirical constant	CPI	η (MFU)

The Iron Law: Every Wall Maps to a Term

$$T = \frac{\text{FLOPs}}{N \times \text{Peak} \times \text{MFU} \times \eta_{\text{scaling}} \times \text{Goodput}}$$

Term	**Walls**
FLOPs (numerator)	11 Complexity 12 Reasoning 13 Fidelity
N (parallelism)	14 Communication 15 Fragility

Term	**Walls**
Peak (hardware)	1 Compute 2 Memory 6 Streaming
MFU (efficiency)	3 Software 4 Serving

22 Walls at a Glance

#	Wall	Domain	Constraint	Key Equation
1	Compute	Node	Peak FLOPS ceiling	$T = \text{OPs}/(\text{Peak} \times \eta)$
2	Memory	Node	HBM capacity & bandwidth	$T = \lvert W \rvert / \text{BW}$
3	Software	Node	Peak vs achieved FLOPS	η_{MFU}
4	Serving	Node	Prefill vs decode regimes	TTFT, ITL
5	Batching	Node	KV cache fragmentation	PagedAttention
6	Streaming	Node	Injection BW (wafer-scale)	$T = \lvert W \rvert / \text{BW}_{\text{inj}}$
7	Tail Latency	Node	P99 grows near saturation	Erlang-C
8	Ingestion	Data	Storage I/O supply rate	$\rho = \text{BW}_d / \text{BW}_s$
9	Transform	Data	CPU preprocessing rate	$T = BS/C$
10	Locality	Data	Bisection bandwidth limit	BW_{eff}
11	Complexity	Algorithm	Chinchilla scaling laws	$C = 6PD$
12	Reasoning	Algorithm	Inference-time compute	$T = K \times T_{\text{step}}$
13	Fidelity	Algorithm	Compression–accuracy tradeoff	$r = 32/b$
14	Communication	Fleet	AllReduce synchronization	Ring AllReduce
15	Fragility	Fleet	Component failures at scale	MTBF/N

The Hardware Zoo: Every Platform in mlsysim

Vendor	Cloud / Training	Edge / Inference	TinyML
NVIDIA	A100, H100, H200, B200	Jetson Orin NX, Orin Nano	—
AMD	MI250X, MI300X	—	—
Intel	Gaudi 2, Gaudi 3	—	—
Google	TPU v4, TPU v5e	Coral Edge TPU	—
AWS	Trainium2	Inferentia2	—
Cerebras	CS-2 (wafer-scale)	—	—
MCU	—	—	ESP32-S3, nRF52840
Custom	Any accelerator via `mlsysim.Hardware.from_spec({...})`		

5 vendors, 20+ platforms, TinyML to Frontier.

mlsysim is a multi-vendor analytical engine, not an NVIDIA wrapper.

Try It Now

One install. No GPU. Sub-second answers.

```
pip install mlsysim
```

```python
import mlsysim

# Profile Llama-3 8B on H100 in one line
profile = mlsysim.Engine.solve(
    mlsysim.Models.Language.Llama3_8B,
    mlsysim.Hardware.H100,
    batch_size=1, seq_len=2048
)
```

Roadmap: You Are Here

Time	Part	Status
9:00–9:30	Part 0: Welcome & Setup	✓ Done
9:30–10:30	**Part 1: Iron Law & Roofline**	← **You are here**
10:45–11:45	Part 2: Memory Walls & Serving	
11:45–12:00	Part 3: Compression	
12:00–1:00	*Lunch*	
1:00–2:15	Part 4: Going Distributed	
2:30–3:15	Part 5: Economics & Sustainability	
3:15–3:45	Part 6: Design Space Exploration	
3:45–4:15	Part 7: TinyML to Frontier	
4:15–4:45	Part 8: Advanced Topics	

TUTORIAL | TUTORIAL

Iron Law & Roofline

3

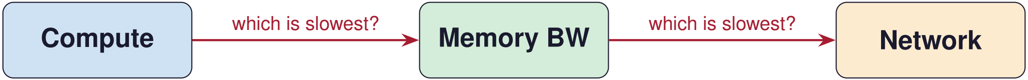
Key Question

Why doesn't doubling FLOPS

double your throughput?

Constraints Drive Architecture

- Hardware has **finite compute** (FLOPS), **finite bandwidth** (GB/s), and **finite memory** (GB).
- Every workload demands some amount of each.
- The **binding constraint** is the one that takes the longest.
- You optimize the bottleneck, not the fast part.



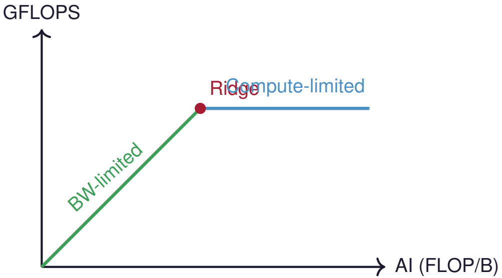
The Roofline Model (Williams et al., 2009)

$$\text{Attainable FLOPS} = \min\left(\text{Peak FLOPS}, \quad \text{BW} \times \text{Arithmetic Intensity}\right)$$

Arithmetic Intensity (AI):

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes moved}} \quad \left[\text{FLOP/byte}\right]$$

- **AI** $<$ **Ridge Point** $\Rightarrow$ Memory-bound
- **AI** $>$ **Ridge Point** $\Rightarrow$ Compute-bound



Ridge Point $=$ Peak FLOPS $/$ Peak BW

Wall 1: The Compute Wall

The Compute Wall

$$T_{\text{compute}} = \frac{\text{Operations}}{\text{Peak FLOPS} \times \text{Efficiency}}$$

Example: ResNet-50 inference at batch 256 on H100

- FLOPs = $8.0 \times 10^9 \times 256 = 2.05 \times 10^{12}$
- H100 FP16 Peak = 989 TFLOPS
- At 50% MFU: $T = \frac{2.05 \times 10^{12}}{989 \times 10^{12} \times 0.5} \approx 4.1$ ms

The chip is the ceiling. MFU is how close you get to it.

Wall 2: The Memory Wall

The Memory Wall

$$T_{\text{memory}} = \frac{\text{Weight Bytes}}{\text{Memory Bandwidth}}$$

Example: Llama-3 8B at batch size 1 on H100

- Weight size (FP16) = 8B $\times$ 2 bytes $=$ 16 GB
- H100 HBM3 BW = 3.35 TB/s
- $T = \frac{16}{3350} \approx$ 4.8 ms just to load weights
- Meanwhile, compute finishes in $\sim$0.03 ms

Predict: H100 vs MI300X vs Gaudi 3

Three flagship accelerators. Same workload.

Llama-3 8B, batch size 1, FP16 inference.

Which is fastest—and by how much?

Predict: H100 vs MI300X vs Gaudi 3

Three flagship accelerators. Same workload.

Llama-3 8B, batch size 1, FP16 inference.
Which is fastest—and by how much?

	H100 (NVIDIA)	MI300X (AMD)	Gaudi 3 (Intel)
Peak FP16	989 TFLOPS	1,307 TFLOPS	1,835 TFLOPS
HBM BW	3.35 TB/s	5.3 TB/s	3.7 TB/s
HBM Capacity	80 GB	192 GB	128 GB
Bottleneck	Memory	Memory	Memory
Weight-load time	4.8 ms	3.0 ms	4.3 ms
Speedup vs H100	—	1.6×	1.1×

Predict: H100 vs MI300X vs Gaudi 3

Three flagship accelerators. Same workload.

Llama-3 8B, batch size 1, FP16 inference.
Which is fastest—and by how much?

	H100 (NVIDIA)	MI300X (AMD)	Gaudi 3 (Intel)
Peak FP16	989 TFLOPS	1,307 TFLOPS	1,835 TFLOPS
HBM BW	3.35 TB/s	5.3 TB/s	3.7 TB/s
HBM Capacity	80 GB	192 GB	128 GB
Bottleneck	Memory	Memory	Memory
Weight-load time	4.8 ms	3.0 ms	4.3 ms
Speedup vs H100	—	1.6×	1.1×

MI300X has fewer FLOPS than Gaudi 3 but wins on bandwidth.

Live Demo: Three Vendors, One API

Run this in your Python session:

```
import mlsysim

model = mlsysim.Models.Language.Llama3_8B
for hw_name in ["H100", "MI300X", "Gaudi3"]:
    hw = getattr(mlsysim.Hardware.Cloud, hw_name)
    p = mlsysim.Engine.solve(model, hw, batch_size=1)
    print(f"{hw.name}: {p.bottleneck}, "
          f"{p.latency:.2f}")
```

What to look for

- `bottleneck`: Memory for **all three**

Wall 3: MFU — The Software Wall

Model FLOPs Utilization

$$\text{MFU} = \frac{\text{Achieved FLOPS}}{\text{Peak FLOPS}}$$

What eats MFU?

- Kernel launch overhead
- Memory stalls (cache misses)
- Framework overhead (Python $\rightarrow$ CUDA)
- Suboptimal operator fusion
- Being memory-bound (the biggest one!)

Typical MFU ranges

Workload	MFU
LLM training (optimized)	40–55%
LLM inference (bs=1)	$<$5%
ResNet training	30–40%
FlashAttention	60–75%

What Is η? (The Efficiency Parameter)

η = **Achieved FLOPS** / **Peak FLOPS** (Model FLOPs Utilization)

The gap between what your hardware *could* do and what it *actually* does.

What reduces η:

- Kernel launch overhead
- SM occupancy limits
- Memory coalescing misses
- Framework overhead (Python GIL)
- Communication stalls

Typical values:

Scenario	η
Training (Megatron-LM)	0.40–0.55
Training (PyTorch eager)	0.08–0.15
Inference decode, bs=1	0.01–0.05
Inference decode, bs=32+	0.15–0.35
Inference prefill	0.30–0.50
TinyML (TFLite Micro)	0.05–0.15

You do not predict η — you measure it once and use it for what-if analysis.

The Iron Law of ML Systems

$$T = \frac{\text{FLOPs}}{N \times \text{Peak} \times \text{MFU} \times \eta_{\text{scaling}} \times \text{Goodput}}$$

Term	Meaning	Reduced by	Walls
N	Number of devices	Budget	—
Peak	Raw hardware speed	GPU generation	1 (Compute)
MFU	Software efficiency	FlashAttention, fusion	2–3
η_{scaling}	Communication loss	BW, gradient compression	14–16
Goodput	Failure overhead	Checkpointing, FT	15, 19

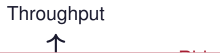
Every wall in the taxonomy attacks one of these five terms

Arithmetic Intensity: The Dial You Control

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes}} \approx \frac{2 \times \text{Params} \times B}{\underbrace{\text{Params} \times \text{bpp}}_{\text{weights}} + \underbrace{\text{Activations}(B)}_{\text{grows with } B}}$$

The batch-size knob:

- At $B = 1$: AI $\approx$ 1 FLOP/byte $\Rightarrow$ **memory-bound**
- At $B = 32$: AI $\approx$ 32 FLOP/byte $\Rightarrow$ approaching ridge
- At $B = 256$: AI $\gg$ ridge $\Rightarrow$ **compute-bound**



Live Demo: Finding the Ridge Point

```python
llama = mlsysim.Models.Language.Llama3_8B
hw    = mlsysim.Hardware.Cloud.H100

for bs in [1, 4, 16, 64, 128, 256]:
    p = mlsysim.Engine.solve(llama, hw, batch_size=bs)
    print(f"bs={bs:>3d}  {p.bottleneck:<8s}  "
          f"MFU={p.mfu:.3f}")
```

What to observe

- Bottleneck flips from `Memory` to `Compute`
- MFU climbs as batch size increases (better hardware utilization)

Exercise 1: Find the Crossover

Hands-On Exercise

Question: At what batch size does Llama-3 8B on H100 transition from memory-bound to compute-bound?

```python
for bs in range(1, 129):
    p = mlsysim.Engine.solve(llama, hw, batch_size=bs)
    if p.bottleneck == "Compute":
        print(f"Crossover at batch size {bs}")
        break
```

Bonus: Try the same on A100. Does the crossover happen at the same batch size?

The Ridge Point: Hardware DNA

$$\text{Ridge Point} = \frac{\text{Peak FLOPS}}{\text{Peak BW}} \quad [\text{FLOP/byte}]$$

Vendor	Hardware	Peak FP16	HBM BW	Ridge
NVIDIA	H100 SXM	989 TFLOPS	3.35 TB/s	295 FLOP/B
NVIDIA	B200	2.25 PFLOPS	8.0 TB/s	281 FLOP/B
AMD	MI300X	1,307 TFLOPS	5.3 TB/s	247 FLOP/B
Intel	Gaudi 3	1,835 TFLOPS	3.7 TB/s	496 FLOP/B

- Higher ridge $\Rightarrow$ more workloads are memory-bound on this chip
- FLOPS grow faster than bandwidth across GPU generations
- The memory wall is getting **worse**, not better

Predict: ResNet-50 vs Llama-3 8B

Two workloads on the same H100.

Which is compute-bound? Which is memory-bound?

Predict: ResNet-50 vs Llama-3 8B

Two workloads on the same H100.

Which is compute-bound? Which is memory-bound?

	ResNet-50 (bs=256)	Llama-3 8B (bs=1)
Total FLOPs	2.05×10^{12}	1.6×10^{10}
Weight bytes	**50 MB (FP16)**	**16 GB (FP16)**
AI (FLOP/B)	**$\sim$41,000**	**$\sim$1**
Regime	**Compute-bound**	**Memory-bound**

Predict: ResNet-50 vs Llama-3 8B

Two workloads on the same H100.

Which is compute-bound? Which is memory-bound?

	ResNet-50 (bs=256)	Llama-3 8B (bs=1)
Total FLOPs	2.05×10^{12}	1.6×10^{10}
Weight bytes	50 MB (FP16)	16 GB (FP16)
AI (FLOP/B)	$\sim$41,000	$\sim$1
Regime	Compute-bound	Memory-bound

Same hardware, completely different bottlenecks.

Part 1: Key Takeaway

The bottleneck determines the speedup.

- The Roofline model tells you *which* constraint is binding.
- Batch size is the primary knob that moves you between regimes.
- More FLOPS only helps if you are compute-bound.
- More bandwidth only helps if you are memory-bound.
- `Engine.solve()` answers this in one line.

Roadmap: You Are Here

Time	Part	Status
9:00–9:30	Part 0: Welcome & Setup	✓
9:30–10:30	Part 1: Iron Law & Roofline	✓
10:45–11:45	**Part 2: Memory Walls & Serving**	← **You are here**
11:45–12:00	Part 3: Compression	

TUTORIAL | TUTORIAL

Memory Walls & Serving

4

Key Question

Why does the first token take 50 ms

but each next token only takes 5 ms?

Wall 4: Prefill vs Decode — Two Different Physics

The Serving Wall

$$\textbf{TTFT}\text{ (Prefill)} = \frac{\text{Prefill FLOPs}}{\text{Peak FLOPS} \times \text{MFU}} \quad \text{Compute-bound}$$

$$\textbf{ITL}\text{ (Decode)} = \frac{\text{Weight Bytes}}{\text{Bandwidth}} \quad \text{Memory-bound}$$

Prefill (process the prompt)

- All prompt tokens in parallel
- $O(S^2)$ attention + $O(S \cdot P)$ linear
- Compute-bound (high AI)
- Determines **TTFT**

Decode (generate tokens)

- One token at a time
- Must reload all weights per token
- Memory-bound (AI $\approx$ 1)
- Determines **ITL**

Wall 5: The KV Cache — Hidden Memory Consumer

The Batching Wall

$$\text{KV cache} = 2 \times L \times H \times d \times S \times B \times \text{bpp}$$

Llama-3 8B at 4K context, FP16:

- $2 \times 32 \times 32 \times 128 \times 4096 \times 2$ bytes ≈ 2 GB per request
- H100 has 80 GB HBM — model weights take 16 GB
- Remaining 64 GB $\div$ 2 GB/request $= \sim$**32 concurrent requests**

The serving paradox

You want high batch size (for throughput) but KV cache limits how many requests fit in memory.

Live Demo: Two-Phase Serving Analysis

```python
serving = mlsysim.ServingModel()
result  = serving.solve(llama, hw,
                        seq_len=4096, batch_size=1)
print(f"TTFT: {result.ttft:~P}")
print(f"ITL:  {result.itl:~P}")
print(f"KV cache/req: {result.kv_cache_per_request:~P}")
```

Expected output

TTFT ≈ 20–50 ms (compute-bound), ITL ≈ 5 ms (memory-bound), KV cache per request ≈ 2 GB

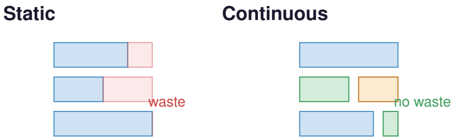
Continuous Batching: Don't Wait, Serve

Static batching

- Pad all sequences to max length
- GPU idle while short requests finish
- Throughput limited by longest request
- Simple to implement

Continuous batching

- Insert new requests per iteration
- No padding waste
- Throughput 2–5$\times$ higher
- Used by vLLM, TGI, TensorRT-LLM



PagedAttention: Virtual Memory for KV Cache

The problem: Pre-allocated KV cache wastes memory on short sequences.

The solution (Kwon et al., 2023 — vLLM):

- Divide KV cache into fixed-size **pages** (e.g., 16 tokens each)
- Allocate pages on demand, not up front
- Non-contiguous storage eliminates fragmentation
- Memory utilization improves from $\sim$50% to $>$95%

Impact

With the same 80 GB H100, PagedAttention can serve **2–4$\times$ more concurrent requests** than static allocation.

Predict: How Many Concurrent Requests?

H100 (80 GB), Llama-3 8B (FP16), 4K context.

How many concurrent requests can you serve?

Predict: How Many Concurrent Requests?

H100 (80 GB), Llama-3 8B (FP16), 4K context.

How many concurrent requests can you serve?

	Static alloc.	PagedAttention
KV cache utilization	∼50%	∼95%
Effective memory/req	∼4 GB	∼2.1 GB
Max concurrent	∼16	∼30

Predict: How Many Concurrent Requests?

H100 (80 GB), Llama-3 8B (FP16), 4K context.

How many concurrent requests can you serve?

	Static alloc.	PagedAttention
KV cache utilization	∼50%	∼95%
Effective memory/req	∼4 GB	∼2.1 GB
Max concurrent	∼16	∼30

PagedAttention nearly doubles serving capacity without changing hardware.

Speculative Decoding: Betting on the Draft

Insight: Decode is memory-bound $\Rightarrow$ the GPU has spare compute.

Speculative decoding (Leviathan et al., 2023):

1. **Draft** K tokens with a small model (fast, low quality)
2. **Verify** all K tokens in one forward pass of the big model
3. **Accept** the longest prefix that matches
4. Speedup $\approx K \times \alpha$ where α is acceptance rate

```
# mlsysim supports speculative decoding
draft = mlsysim.Models.Language.Llama3_8B  # smaller
result = serving.solve(model, hw, seq_len=4096,
              draft_model=draft, draft_acceptance_rate=0.7)
```

Disaggregated Serving: Right Hardware for Each Phase

Key insight: Prefill and decode have *opposite* hardware preferences.

Prefill node

- Needs high FLOPS
- Moderate memory
- E.g., H100 at high utilization

Decode node

- Needs high BW
- Large memory (for KV cache)
- E.g., many smaller accelerators

```
# Disaggregated serving in mlsysim
result = serving.solve(model, hw, seq_len=4096,
          decode_hardware=mlsysim.Hardware.Cloud.A100)
print(f"TTFT: {result.ttft:~P}  ITL: {result.itl:~P}")
```

Trade-off: network transfer of KV cache adds latency between phases

Exercise 2: Serving Capacity Planning

Hands-On Exercise

Question: You run Llama-3 8B (FP16) on one H100 with 4K context.
Your SLA requires ITL $<$ 10 ms.
How many concurrent requests can you serve?

```python
cb = mlsysim.ContinuousBatchingModel()
result = cb.solve(llama, hw, seq_len=4096,
                  max_batch_size=64, page_size=16)
print(f"Max concurrent: {result.max_concurrent_requests}")
```

Bonus: What happens if you switch to INT8 precision? Does concurrency double?

Fallacies: Serving Edition

Fallacy: *"Faster GPUs always reduce latency."*
A 3.2$\times$ FLOPS improvement (A100 $\rightarrow$ H100) yields only 1.7$\times$ ITL improvement because decode is memory-bound.

Fallacy: *"Doubling memory doubles serving capacity."*
Weights are fixed overhead. Going from 80 GB to 160 GB adds 80 GB, but KV cache per request stays $\sim$2 GB. Capacity goes from $\sim$32 to $\sim$72 ($\sim$2.3$\times$), not 2$\times$.

Fallacy: *"Batch size 1 is fine for LLM inference."*
At bs=1, MFU $<$ 5%. You are paying for 989 TFLOPS and using $<$ 50. Continuous batching can recover 10–20$\times$ throughput.

Part 2: Key Takeaway

LLM serving has two phases with opposite bottlenecks.

- **Prefill** (TTFT) is compute-bound — optimize with parallelism.
- **Decode** (ITL) is memory-bound — optimize with bandwidth.
- **KV cache** limits concurrency — optimize with PagedAttention.
- `ServingModel.solve()` decomposes both phases in one call.

TUTORIAL | TUTORIAL

Compression & Efficiency

5

Key Question

Can you make the model 4× smaller

and get a 4× speedup?

Wall 13: The Fidelity Wall

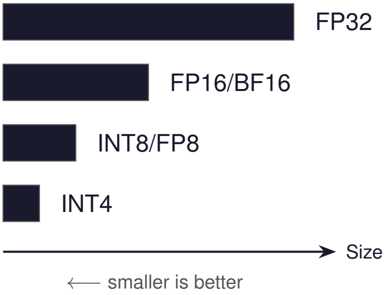
The Fidelity Wall

$$\text{Compression}_{\text{quant}} = \frac{32}{\text{bits}}, \qquad \text{Compression}_{\text{prune}} = \frac{1}{1 - \text{sparsity}}$$

Method	**Storage**	**Speedup**	**Accuracy**
FP32 $\rightarrow$ FP16	$2\times$	$2\times$	$\sim$0% loss
FP16 $\rightarrow$ INT8	$2\times$	1.5–$2\times$	$<$1% loss
FP16 $\rightarrow$ INT4	$4\times$	2–$3\times$	2–5% loss
50% unstructured prune	$2\times$	$1\times$ (no speedup!)	1–3% loss
50% structured prune	$2\times$	$\sim 2\times$	2–5% loss
2:4 N:M sparsity	$2\times$	$2\times$	1–2% loss

Quantization: Trading Bits for Speed

The precision ladder:



Llama-3 8B memory:

Precision	Weight Size
FP32	32 GB
FP16	16 GB
INT8	8 GB
INT4	4 GB

At INT4, Llama-3 8B fits in a **laptop GPU** (6 GB).

Live Demo: Quantization Impact

```
comp = mlsysim.CompressionModel()
for bits in [16, 8, 4]:
    r = comp.solve(llama, hw, method="quantization",
                   target_bitwidth=bits)
    print(f"INT{bits}: size={r.compressed_size:~P}  "
          f"speedup={r.inference_speedup:.1f}x")
```

What to observe

- Storage shrinks linearly with bit reduction
- Speedup follows storage for quantization (structured by nature)
- Accuracy degrades modestly at INT8, more at INT4

Pruning: The Structure Matters

Unstructured

- Zero out individual weights
- Matrix shape unchanged
- GPU does same work (skip zeros? nope)
- Storage savings only

Structured / N:M

- Remove entire rows/columns, or 2:4 pattern (Ampere+)
- Physically smaller matrices
- GPU hardware support (2:4 $\rightarrow$ 2$\times$)
- **Real speedup**

```
for stype in ["unstructured", "structured", "n_m"]:
    r = comp.solve(llama, hw, method="pruning",
            sparsity=0.5, sparsity_type=stype)
    print(f"{stype:>14}: {r.inference_speedup:.1f}x")
```

Predict: What Does INT4 Change?

You quantize Llama-3 70B from FP16 to INT4.

What is the BIGGEST impact on your serving infrastructure?

(A) Inference latency drops by $4\times$

(B) Model quality degrades significantly

(C) **You need half as many GPUs**

(D) Memory bandwidth becomes the bottleneck

Predict: INT4 Llama-3 8B on H100

Quantize Llama-3 8B to INT4.

Run inference at batch size 1 on H100.

Is it still memory-bound?

Predict: INT4 Llama-3 8B on H100

Quantize Llama-3 8B to INT4.
Run inference at batch size 1 on H100.

Is it still memory-bound?

	FP16	INT4
Weight size	16 GB	4 GB
Load time	4.8 ms	1.2 ms
Compute time	0.03 ms	0.03 ms
Bottleneck	Memory	Still Memory!
Decode speedup	—	$4\times$

Predict: INT4 Llama-3 8B on H100

Quantize Llama-3 8B to INT4.
Run inference at batch size 1 on H100.

Is it still memory-bound?

	FP16	INT4
Weight size	16 GB	4 GB
Load time	4.8 ms	1.2 ms
Compute time	0.03 ms	0.03 ms
Bottleneck	Memory	Still Memory!
Decode speedup	—	4×

Exercise 3: Compression Tradeoffs

Hands-On Exercise

Question: For Llama-3 8B on H100, which is the better deal?

1. **INT8 quantization**
2. **50% structured pruning**

Compare speedup per accuracy point lost.

```
r1 = comp.solve(llama, hw, method="quantization",
                target_bitwidth=8)
r2 = comp.solve(llama, hw, method="pruning",
        sparsity=0.5, sparsity_type="structured")
print(f"Quant: {r1.inference_speedup:.1f}x / "
```

Compression Changes Fleet Architecture

Llama-3 70B Serving Fleet:

Precision	Model Size	GPUs Needed	Annual Cost
FP16	140 GB	4 (TP=4)	$480K
INT8	70 GB	2 (TP=2)	$240K
INT4	35 GB	1	$120K

INT4 doesn't just improve latency — it eliminates 3 GPUs per replica.
At 100 replicas for 1000 QPS: that's **300 fewer GPUs** and **$36M saved per year**.

This is why quantization is a Day 1 architectural decision, not a Day 100 optimization.

Part 3: Key Takeaway

Storage savings $\neq$ inference speedup.

- Quantization gives both storage and speed gains.
- Unstructured pruning gives storage only — zero GPU speedup.
- N:M sparsity (2:4) is the hardware-friendly middle ground.
- Even at INT4, LLM decode is *still* memory-bound.
- `CompressionModel.solve()` quantifies the full tradeoff.

Roadmap: Afternoon Session

Time	Part	Status
9:00–12:00	Parts 0–3: Single Node	✓ Done
1:00–2:15	**Part 4: Going Distributed**	← **You are here**
2:30–3:15	Part 5: Economics & Sustainability	
3:15–3:45	Part 6: Design Space Exploration	
3:45–4:15	Part 7: TinyML to Frontier	
4:15–4:45	Part 8: Advanced Topics	
4:45–5:00	Part 9: Wrap-Up & Capstone	

TUTORIAL | TUTORIAL

Going Distributed

6

Key Question

If 1 GPU takes 30 days,

do 1000 GPUs take 43 minutes?

Why Distribute?

Reason 1: Model does not fit

- Llama-3 70B FP16 $=$ 140 GB $>$ H100's 80 GB
- Must split across at least 2 GPUs

Reason 2: Time-to-train

- 1 H100 training Llama-3 70B $\approx$ 15 GPU-years
- 1024 H100s $\approx$ 5 days (if scaling were perfect)
- But scaling is *never* perfect...

Speedup

3D Parallelism: DP $\times$ TP $\times$ PP

Data Parallel (DP)

- Replicate full model
- Split data across replicas
- AllReduce gradients
- *Most common*

Tensor Parallel (TP)

- Split each layer's weights
- Split activations, not data
- AllReduce per layer ($2\times$!)
- Needs fast interconnect

Pipeline Parallel (PP)

- Split model into stages
- Each GPU owns L/PP layers
- Pipeline bubbles
- Needs less bandwidth

Total GPUs: $N = \text{DP} \times \text{TP} \times \text{PP}$

Property	DP	TP	PP
Splits	Data	Weights + Activations	Layers
Communication	AllReduce (gradients)	AllReduce (activations)	Point-to-point
BW requirement	Moderate	Very high (NVLink)	Low

AllReduce: A Concrete Example

Setup: 8 H100 GPUs, NVLink at 900 GB/s, Llama-3 8B (16 GB gradients)

1. Each GPU computes its local gradient: **16 GB**
2. All 8 GPUs must end up with the **same averaged gradient**
3. Ring AllReduce passes chunks around the ring...

```
t = mlsysim.core.formulas.calc_ring_allreduce_time(
    message_bytes=16e9,
    n_gpus=8,
    bandwidth_bytes_s=900e9,
    latency_s=500e-9,
)
```

Wall 14: The Communication Wall (AllReduce)

The Communication Wall

$$T_{\text{AllReduce}} = \frac{2(N-1)}{N} \times \frac{M}{BW} + 2(N-1) \times \text{latency}$$

Example: 1 GB gradients, 8× H100 on NVLink (900 GB/s)

$$T = \frac{2 \times 7}{8} \times \frac{1}{900} \approx 1.9 \text{ ms}$$

Same gradients, 256× H100 across InfiniBand (50 GB/s):

$$T = \frac{2 \times 255}{256} \times \frac{1}{50} \approx 40 \text{ ms}$$

Tensor Parallelism: Splitting Layers

How it works:

1. Split weight matrix W column-wise across T GPUs
2. Each GPU computes $Y_i = X \cdot W_i$ (partial result)
3. AllReduce to combine: $Y = \sum Y_i$
4. 2 AllReduce ops per layer (forward + backward)

TP overhead:

$$T_{\mathrm{TP}} = 2 \times L \times T_{\mathrm{AllReduce}}(T)$$

Llama-3 70B, TP=8 on NVLink (900 GB/s)

Pipeline Parallelism: The Bubble Problem

Pipeline Bubble Fraction

$$\text{Bubble} = \frac{P-1}{M+P-1}$$

where P = pipeline stages, M = microbatches

P (stages)	M (microbatches)	Bubble	Effective utilization
4	4	43%	57%
4	16	16%	84%
4	32	8.6%	91%
8	32	18%	82%

More microbatches $\Rightarrow$ smaller bubble.

But more microbatches = more memory for activations.

Gradient Accumulation: Virtual Batch Size

$$B_{\text{effective}} = B_{\text{micro}} \times K \times \text{DP}$$

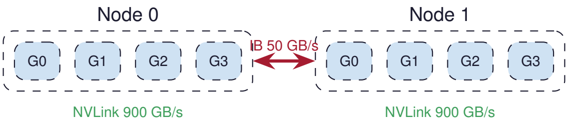
Why accumulate?

- Fill the pipeline ($M = K$ microbatches)
- Amortize AllReduce cost over K steps
- Simulate large batch size without large memory
- Trade compute (more forward passes) for communication (fewer AllReduce)

Example: DP=128, B_{micro}=4, K=8

Hierarchical AllReduce: NVLink + InfiniBand

Real cluster topology:



3-step hierarchical AllReduce:

1. **Local reduce** within each node (NVLink — fast)
2. **Global AllReduce** across leader GPUs (InfiniBand — slow)
3. **Local broadcast** within each node (NVLink — fast)

TP within node (NVLink), DP across nodes (InfiniBand)

Live Demo: Distributed Training Analysis

```python
fleet  = mlsysim.Systems.Clusters.Frontier_8K
dist   = mlsysim.DistributedModel()
result = dist.solve(llama, fleet, batch_size=4096,
         tp_size=8, pp_size=1, microbatch_count=32,
         seq_len=4096)
print(f"Scaling eff:    {result.scaling_efficiency:.1%}")
print(f"Comm overhead: {result.communication_overhead:.1%}")
print(f"Effective MFU: {result.effective_mfu:.1%}")
```

What to look for

Communication overhead + bubble fraction = total efficiency loss. Effective MFU = single-node MFU × scaling efficiency.

Wall 15: The Fragility Wall (Reliability)

The Fragility Wall

$$\text{Cluster MTBF} = \frac{\text{Component MTBF}}{N_{\text{components}}}$$

Scale	GPUs	Cluster MTBF
Research lab	8	260 days
Mid cluster	256	8 days
Large cluster	1,024	2 days
Frontier-scale	8,192	6 hours
Mega cluster	100K	30 minutes

At frontier scale, something breaks every 6 hours.

Predict: Scaling to 256 GPUs

You have 8 H100s doing data-parallel training.

You scale to 256 GPUs.

How much faster will training be?

(A) 32× faster (perfect scaling)

(B) 20–25× faster

(C) 10–15× faster

(D) **It depends on the model size**

Scaling Efficiency: The Amdahl Trap

$$\eta_{\text{scaling}} = \frac{\text{Actual speedup}}{N} = \frac{1}{1 + \text{comm\_frac} + \text{bubble\_frac} + \text{straggler\_frac}}$$

What eats scaling efficiency:

1. AllReduce communication
2. Pipeline bubbles
3. Straggler effects (slowest GPU)
4. Checkpoint I/O
5. Failure recovery

System	η_{scaling}
8 GPUs (NVLink)	95–98%
64 GPUs (IB)	85–92%
1024 GPUs	70–85%
8192 GPUs	55–70%

At 8192 GPUs, you lose 30–45% of your compute to overhead

Predict: Optimal Parallelism Config

64 H100s. Llama-3 70B (140 GB FP16).

What is the optimal TP $\times$ PP $\times$ DP?

Predict: Optimal Parallelism Config

64 H100s. Llama-3 70B (140 GB FP16).

What is the optimal TP $\times$ PP $\times$ DP?

Config	TP	PP	DP	Why
Candidate A	8	1	8	TP within node, no bubbles
Candidate B	4	2	8	Less TP comm, but has bubbles
Candidate C	2	4	8	Minimal TP, large bubble

Predict: Optimal Parallelism Config

64 H100s. Llama-3 70B (140 GB FP16).

What is the optimal TP $\times$ PP $\times$ DP?

Config	TP	PP	DP	Why
Candidate A	8	1	8	TP within node, no bubbles
Candidate B	4	2	8	Less TP comm, but has bubbles
Candidate C	2	4	8	Minimal TP, large bubble

Candidate A is typically best: TP=8 uses full NVLink bandwidth, PP=1 avoids pipeline bubbles entirely, DP=8 across nodes.

Exercise 4: Distributed Training Design

Hands-On Exercise

Question: Find the optimal TP$\times$PP for Llama-3 70B on 64 H100s.

```
llama70 = mlsysim.Models.Language.Llama3_70B
fleet   = mlsysim.Systems.Clusters.Research_256
for tp in [1, 2, 4, 8]:
    for pp in [1, 2, 4, 8]:
        if tp * pp > 64: continue
        r = dist.solve(llama70, fleet, batch_size=512,
            tp_size=tp, pp_size=pp, seq_len=4096,
            microbatch_count=max(4, 64//(tp*pp)))
        print(f"TP={tp} PP={pp} eff={r.scaling_efficiency:.1%}")
```

Stragglers: The Slowest GPU Sets the Pace

Synchronous training: step time $= \max_i(T_i)$

- **Thermal throttling:** hot GPUs clock down 5–10%
- **Network congestion:** some AllReduce messages delayed
- **OS jitter:** background tasks steal cycles
- **Memory pressure:** GC pauses in the data pipeline

Mitigation strategies:

- Asynchronous SGD (trade accuracy for speed)
- Backup workers (redundant computation)

Part 4: Key Takeaway

Distributed training is a communication problem disguised as a compute problem.

- 3D parallelism (DP $\times$ TP $\times$ PP) decomposes the problem.
- TP needs NVLink (within node). DP works over InfiniBand (across nodes).
- Pipeline bubbles shrink with more microbatches.
- Reliability degrades as MTBF$/N$ — checkpointing is mandatory.
- `DistributedModel.solve()` captures all these effects.

Lunch break — reconvene at 1:00 PM for Part 5.